

NATIONAL RESEARCH UNIVERSITY HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "HSE and University of London Double Degree Programme in Data
Science and Business Analytics"

Programming project report

on the topic Numerical Algorithms of Linear Algebra

Student:

group БПА/Д232

Sign

З. Ю. Зинкин

Name, Patronymic, Surname

4.02.2025

Date

Supervisor:

Дмитрий Витальевич Трушин

Name, Patronymic, Surname

доцент / ФКН НИУ ВШЭ

Position / Company

4.02.2025

Date

Grade

Sign

Moscow 2025

Contents

1	Introduction	3
2	Description of functional and non-functional requirements for the program project	4
2.1	Functional requirements	4
2.2	Non-functional requirements	4
3	Theoretical part	5
3.1	Main definitions	5
3.2	Algorithms	7
3.2.1	Householder reflection	7
3.2.2	Givens rotations	8
3.2.3	QR decomposition	9
4	Library LinAlgTools	11
4.1	Types	11
4.2	Core	12
4.3	Algorithms	12
5	Testing	13
6	Conclusion	14

Abstract

The main focus of the programming project is to study matrix decomposition algorithms from a theoretical point of view with its subsequent implementation on the C++ language.

1 Introduction

Decomposition algorithms are extensively used in Numerical Linear Algebra and Computer Science. They can help store large matrices, find eigenvalues of matrices, solve systems of linear equations and linear least squares problems and much more. This is why it is vital not only to recognize the potential use cases of decomposition algorithms but also understand the underlying theory and implementation of such algorithms.

The goal of my programming project is to learn modern algorithms of Numerical Linear Algebra with a focus on matrix decomposition algorithms, which represent a matrix in a certain way, as a product of other matrices, for further computation. In order to have a profound understanding of each of the algorithms, it is necessary to implement a C++ library from scratch.

C++ is the language of choice for the project, since it provides high performance through low-level memory control and compiler optimization. Moreover, its specialization in object-oriented design and template system make it suitable for working with matrices.

The library contains a class for matrix representation with implementation of matrix arithmetic and other necessary methods. Decomposition algorithms are implemented based on the aforementioned matrix class. The whole library is tested using Googletest[2].

The information for the theoretical part of the project is taken from the books, which are cited at the end of the paper. Citation of exact chapters is used throughout the theoretical part when needed.

The result of the project is a C++ library `LinAlgTools`[3] with the following functionality:

- Matrix arithmetic
- QR Decomposition
- Singular Value Decomposition (SVD)

Theoretical part can help you understand the underlying theory of each of the algorithms. The technical part of the project is described in detail in the Library `LinAlgTools` section of the paper. The results of the testing using Googletest[2] are documented in the Testing section.

2 Description of functional and non-functional requirements for the program project

2.1 Functional requirements

- A *Matrix* class, which allows users to perform matrix arithmetic. The class must be able to support both real and complex elements. Moreover, the class must have a clear and intuitive interface.
- A *SubMatrix* and *ConstSubMatrix* classes for handling a particular part of an existing matrix without copying data. These classes must have the same methods as the *Matrix* class in order for users to be able to work with them in the same manner, as with regular matrices. Furthermore, cross-type arithmetic must be implemented to perform arithmetic between objects of *Matrix* and *SubMatrix* classes of appropriate sizes.
- Implementation of numerically stable algorithms:
 - QR Decomposition using Householder rotations - representation of a matrix as a product of Q and R matrices, which are computed using Householder rotations.
 - QR Decomposition using Given's rotations - representation of a matrix using specific Q and R matrices, which are computed using Givens rotations.
 - SVD Decomposition - representation of a matrix as a product of U , Σ and V^* matrices, where U , V are unitary and Σ is a diagonal matrix.
- Extensive testing must be done to guarantee proper functioning of the library. Randomized performance tests are to be analyzed.

2.2 Non-functional requirements

- Implementation of the library on the C++ language without third-party libraries, except for Googletest.
- *Git*, a version control system, and a remote repository on *GitHub* [3].
- *clang* compiler with C++23 standart support.
- Code formatter *clang-format* to automatically format code based on the Microsoft style with additional changes.
- Static code analyzer *clang-tidy* to identify potential errors and to stick to uniform naming of variables, functions, etc.
- A third-party library *Googletest* [2] for writing tests.
- A text editor with IDE capabilities *Neovim*
- Minimum RAM requirement: 4GB.

3 Theoretical part

Theoretical part is split into two parts: *Definitions* and *Algorithms*. In the former part I provide the necessary definitions to establish a rigorous framework for the algorithm discussed later. In the latter part I describe implemented algorithms from a theoretical point of view by providing verbal description, mathematical statements with proofs and pseudocode. I make a particular emphasis on proofs, since they not only mathematically validate the algorithms but often reveal the their structure. Pseudocode sections deliberately avoid edge-cases to provide simplicity.

3.1 Main definitions

Definition 1. An $m \times n$ matrix over a field $\mathbb{F}$ (either $\mathbb{C}$ or $\mathbb{R}$) is a rectangular array of elements of $\mathbb{F}$, arranged in m rows and n columns:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

Remark. A set of all $m \times n$ matrices over a field $\mathbb{F}$ we denote $\mathbb{F}^{m \times n}$.

Definition 2. A matrix $A \in \mathbb{F}^{m \times n}$ is called *diagonal* if all its elements not on the diagonal are 0:

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & 0 \end{bmatrix}$$

Remark. A special case of a diagonal matrix is an $n \times n$ *Identity* matrix:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Definition 3. A matrix $A \in \mathbb{F}^{m \times n}$ is called *upper triangular* if all its elements below the diagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1k} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2k} & \dots & a_{2n} \\ 0 & 0 & a_{33} & \dots & a_{3k} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & a_{mn} \end{bmatrix}$$

Definition 4. A vector is an $1 \times n$ or $n \times 1$ matrix:

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \text{ or } [a_1 \ a_2 \ \dots \ a_n]$$

Remark. A set of all $m \times 1$ or $1 \times m$ vectors over $\mathbb{F}$ we denote $\mathbb{F}^m$.

Definition 5. A *Hermitian conjugate* of a matrix $A \in \mathbb{C}^{m \times n}$ is an $n \times m$ matrix, obtained by transposing A and applying complex conjugate to each of entry: $(A^*)_{ij} = \overline{A_{ji}}$.

Remark. For real matrices *Hermitian conjugate* coincides with transposition of the matrix: $A^* = A^T$.

Definition 6. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Hermitian* if $A^* = A$.

Definition 7. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Unitary* if $A^* = A^{-1}$. As a consequence, $A^*A = AA^* = I$.

Definition 8. A matrix $A \in \mathbb{R}^{m \times n}$ is called *Orthogonal* if $A^T = A^{-1}$. As a consequence, $A^TA = AA^T = I$.

Statement 1. A product of unitary (orthogonal) matrices is also unitary (orthogonal).

Proof. Let U and V be unitary:

$$(UV)^*UV = V^*U^*UV = V^*V = I$$

□

Definition 9. A scalar product is a bilinear form $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^m \rightarrow \mathbb{F}$. In a Euclidean space, such as $\mathbb{R}^m$ or $\mathbb{C}^m$ the scalar product of two vector $v, w \in \mathbb{C}^m$ is defined in the following way:

$$\langle v, w \rangle = w^*v = \sum_{i=1}^m v_i \bar{w}_i$$

Definition 10. A 2-norm, or Euclidean norm, on an Euclidean space is a function: $\|\cdot\|_2 : \mathbb{F}^n \rightarrow \mathbb{R}_+$.

Let $v \in \mathbb{F}^n$:

$$\|v\|_2 = \sqrt{\langle v, v \rangle} = \sqrt{\sum_{i=1}^m |v_i|^2}$$

Remark. Geometrically, a 2-norm returns the distance from the origin to the point in the space.

Definition 11. A matrix is called *sparse* if the majority of its elements are 0.

Definition 12. A matrix is called *dense* if the majority of its elements are non-zero.

Remark. There is no definite rule to tell whether the matrix is sparse or dense, but the usual practice is to consider a matrix sparse if the number of non-zero elements approximately equals to the number of rows or columns.

Definition 13. The i -th standart basis vector of a vector space $\mathbb{F}^m$ will be denoted $e_i = [0, \dots, \underset{i}{1}, \dots, 0]$

3.2 Algorithms

3.2.1 Householder reflection

A *Householder reflection* is a linear transformation which reflects a vector about a hyperplane.^[1]

Given a vector $v \in \mathbb{F}^m : \|v\| = 1$, an $m \times m$ matrix P (or more commonly $H(v)$) of the form

$$P = H(v) = I - 2vv^*$$

is called a *Householder reflector* or *Householder matrix*. The matrix allows us to reflect vector along the hyperplane, spanned by v . Moreover, a Householder matrix is unitary and hermitian ($H^{-1} = H^* = H$), which turns out to be key in the QR decomposition algorithm.

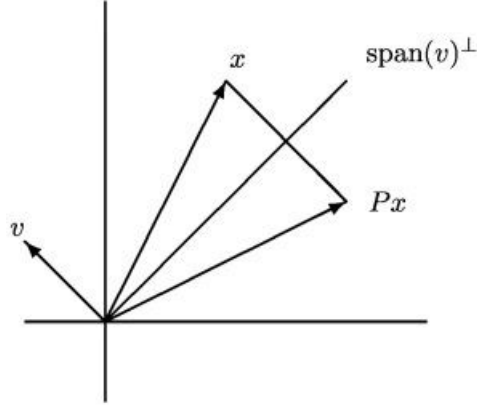


Figure 1: Householder reflection applied to a vector x

Statement 2. $\forall a, b \in \mathbb{C}^n : \|a\|_2 = \|b\|_2, \exists \gamma \in \mathbb{C} : |\gamma| = 1$ and $\exists v \in \mathbb{C}^n : \|v\|_2 = 1$, such that $H(v)a = \gamma b$.

Proof. By direct computation we can find the required γ and v :

$$H(v) = I - 2vv^*$$

$$H(v)a = a - 2vv^*a = \gamma b$$

$$2(v^*a)v = a - \gamma b \tag{1}$$

From here it follows that v must be of the form:

$$v = \mu \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Since we are proving the existence, we can set μ to 1. Therefore

$$v = \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Plugging it into (1):

$$\begin{aligned} 2 \frac{(a - \gamma b)^*}{\|a - \gamma b\|_2} a \frac{a - \gamma b}{\|a - \gamma b\|_2} &= a - \gamma b \\ 2(a - \gamma b)^* a &= \|a - \gamma b\|_2^2 \\ 2(a - \gamma b)^* a &= (a - \gamma b)^*(a - \gamma b) \\ 2a^*a - 2\bar{\gamma}b^*a &= a^*a + b^*b - \gamma a^*b - \bar{\gamma}b^*a \\ \bar{\gamma}b^*a &= \text{Re}(\bar{\gamma}b^*a) \\ \gamma &= \pm \frac{b^*a}{|b^*a|} \end{aligned}$$

If $a \perp b$, γ can be any number, such that $|\gamma| = 1$. If $a \parallel b$, then we should carefully choose the sign to guarantee numeric stability of the transformation. $\square$

Corollary 1. $\forall x \in \mathbb{C}^n \exists v \in \mathbb{C}^n : \|v\|_2 = 1$ and

$$H(v)x = \gamma \begin{bmatrix} \|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$v = \frac{x - \gamma \|x\|_2 e_1}{\|x - \gamma \|x\|_2 e_1\|_2}, \gamma = -\frac{x_1}{|x_1|}$$

The corollary yields an algorithm for transforming a vector, and if we want to apply the transformation to a matrix from the left (right), we can use:

Algorithm 1 Householder Left Reflection

```

1: function HOUSEHOLDERLEFTREFLECTION( $A \in \mathbb{C}^{m \times n}$ ,  $x \in \mathbb{C}^m$ )
2:    $v = x$ 
3:    $v_1 = x_1 - \text{sign}(x_1) \cdot \|x\|_2$ 
4:    $v = v \cdot \frac{1}{\|v\|}$ 
5:    $M = A - (2v) \cdot (v^* A)$ 
6:   return M
7: end function

```

3.2.2 Givens rotations

In case need to zero specific elements of the matrix, *Givens rotations* are our tool of choice. A *Givens rotation* is a linear transformation which amounts to a counterclockwise rotation by an angle of θ . The matrix of a *Givens rotation* is of the form ($c = \cos \theta$, $s = \sin \theta$):

$$G(i, j, \theta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & c & \cdots & -s & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & s & \cdots & c & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{matrix} \\ \\ i \\ \\ j \\ \\ \end{matrix}$$

Statement 3. $\forall v \in \mathbb{F}^m, i, j \in \mathbb{N}_+ \exists \theta \in \mathbb{F} :$

$$G(i, j, \theta) \begin{bmatrix} \vdots \\ a \\ \vdots \\ b \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ r \\ \vdots \\ 0 \\ \vdots \end{bmatrix} \begin{matrix} i \\ j \end{matrix}$$

Proof. If $x \in \mathbb{F}^m$ and

$$y = G(i, j, \theta)x$$

Then:

$$y_j = \begin{cases} cx_i - sx_k, & j = i \\ sx_i + cx_k, & j = k \\ x_j, & j \neq i, k \end{cases}$$

From here we can deduct that $y_k = 0$ if:

$$c = \frac{|x_i|}{\sqrt{|x_i|^2 + |x_k|^2}}, \quad s = \frac{-\bar{x}_k}{\sqrt{|x_i|^2 + |x_k|^2}} \cdot \frac{x_i}{|x_i|}$$

□

Notice that G is almost the identity matrix. We can exploit this structure to achieve faster computation by apply the transformation directly to the rows which are affected by G .

Algorithm 2 Givens Left Rotation

```

1: function GIVENSLEFTROTATION( $A \in \mathbb{F}^{m \times n}$ ,  $i \in \mathbb{N}_+$ ,  $j \in \mathbb{N}_+$ ,  $a \in \mathbb{F}$ ,  $b \in \mathbb{F}$ )
2:    $r = \sqrt{|a|^2 + |b|^2}$ 
3:    $c = a/r$ 
4:    $s = -b/r$ 
5:    $M = A$ 
6:   for  $k = 1$  to  $n$  do
7:      $M_{ik} = \bar{c} \cdot A_{ik} - \bar{s} \cdot A_{jk}$ 
8:      $M_{jk} = s \cdot A_{ik} + c \cdot A_{jk}$ 
9:   end for
10:  return  $M$ 
11: end function

```

3.2.3 QR decomposition

QR decomposition is a tool for solving systems of linear equations and finding eigenvalues of a matrix. The latter is the main reason for its implementation, since it allows to use *QR Algorithm* which will be discussed later. The decomposition stability together with its use in other algorithms make it indispensable in our case.

Theorem 1. For any matrix $A \in \mathbb{F}^{m \times n}$ exist a unitary matrix $Q \in \mathbb{F}^{m \times m}$ and an upper-triangular matrix $R \in \mathbb{F}^{m \times n}$, such that $A = QR$.

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{4 \times 3}$):

$$\begin{aligned}
A = \begin{bmatrix} \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \end{bmatrix} &\xrightarrow{H(v_1)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \end{bmatrix} \xrightarrow{H(v_2)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \end{bmatrix} \xrightarrow{H(v_3)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & 0 \end{bmatrix} = R \\
&H_3 H_2 H_1 A = R \\
A = H_1^{-1} H_2^{-1} H_3^{-1} R = H_1 H_2 H_3 R = QR
\end{aligned}$$

□

Algorithm 3 Householder QR Decomposition

```

1: function HOUSEHOLDERQR( $A \in \mathbb{F}^{m \times n}$ )
   Complexity:  $\mathcal{O}(4mn^2 - \frac{4}{3}n^3)$ 
2:    $Q = I_m$ 
3:    $R = A$ 
4:   for  $k = 1$  to  $n$  do
5:      $\mathbf{x} = R[k : m, k]$ 
6:      $v = \text{HouseholderReduction}(\mathbf{x})$ 
7:      $R[k : m, k : n] = \text{HouseholderLeftReflection}(R[k : m, k : n], v)$ 
8:      $Q[k : m, 1 : m] = \text{HouseholderLeftReflection}(Q[k : m, 1 : m], v)$ 
9:   end for
10:   $Q = Q^T$ 
11:  return  $Q, R$ 
12: end function

```

Proof. Using Givens rotations (for convenience $A \in \mathbb{F}^{3 \times 2}$):

$$A = \begin{bmatrix} \star & \star \\ \star & \star \\ \star & \star \end{bmatrix} \xrightarrow{G_{13}} \begin{bmatrix} \star & \star \\ \star & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{12}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & 0 \end{bmatrix} = R$$

$$G_{23}G_{12}G_{13}A = R$$

$$A = G_{13}^*G_{12}^*G_{23}^*R = QR$$

□

Algorithm 4 Givens QR Decomposition

```

1: function GIVENSQR( $A \in \mathbb{F}^{m \times n}$ )
   Complexity:  $\mathcal{O}(6mn^2 - \frac{5}{2}n^3)$ 
2:    $Q = I_m$ 
3:    $R = A$ 
4:   for  $k = 1$  to  $n$  do
5:     for  $i = m$  downto  $k + 1$  do
6:        $a = R[i - 1, k]$ 
7:        $b = R[i, k]$ 
8:       if  $b \neq 0$  then
9:          $R[1 : m, k : n] = \text{GivensLeftRotation}(R[1 : m, k : n], i - 1, i, a, b)$ 
10:         $Q[1 : m, 1 : m] = \text{GivensRightRotation}(Q[1 : m, 1 : m], i - 1, i, a, b)$ 
11:       end if
12:     end for
13:   end for
14:   return  $Q, R$ 
15: end function

```

While Householder implementation might look superior to Givens due to better complexity, it is not always the case. The main strength of Givens rotations is the ability to zero out specific elements, making it particularly good for sparse matrices or matrices which are close to being upper-triangular. Householder reflections, on the other hand, perform better on dense matrices due to full-column operations.

Therefore each implementation has its own strengths and weaknesses which should be taken into account when working with structured matrices.

4 Library LinAlgTools

The library has the following architecture:



Figure 2: Directory structure of the LinAlgTools library

4.1 Types

This part of the library contains the **Matrix** class and its derivatives: **SubMatrix** and **ConstSubMatrix** classes.

- The **Matrix** class allows to construct matrices of different sizes, with elements of different types (c++ fundamental types or `std::complex`) and perform matrix arithmetic. The class features all the necessary methods for algorithms to work correctly, as well as many others that may be useful in basic linear algebra calculations. Moreover, **concepts** are used to allow for cross-type matrix arithmetic. Hence, it is possible to multiply/add/subtract matrices of different types, for example, an object of a `matrix` class and an object of a `submatrix` class.

```
1 LinAlgTools::Matrix<int> matrix = {{1, 2}, {3, 4}};  
2 std::cout << matrix; //[1, 2], [3, 4]  
3  
4 LinAlgTools::Matrix<int> sq_matrix = {2};  
5 std::cout << sq_matrix; //[0, 0], [0, 0]  
6  
7 LinAlgTools::Matrix<int> identity = LinAlgTools::Matrix<int>::Identity(2);  
8 std::cout << identity; //[1, 0], [0, 1]
```

- The **SubMatrix** and **ConstSubMatrix** classes allow to perform operations with a particular part of the matrix without copying data. This is particularly useful in our case, since we want to apply transformations to a submatrix, which can greatly reduce the number of unnecessary computations. The classes also allow to create an object of a `matrix` class.

```
1 LinAlgTools::Matrix<int> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
2 LinAlgTools::SubMatrix<int> submatrix = {matrix, {0, 1}, {0, 1}};  
3 std::cout << submatrix; //[1, 2], [4, 5]]1  
4  
5 submatrix *= 5;  
6 std::cout << matrix; //[5, 10, 3], [20, 25, 6], [7, 8, 9]];
```

4.2 Core

The entirety of this part of the library is dedicated towards providing useful functions or other functionality, which is used throughout the project.

- **Math_utils** provide the ability to calculate the sign of a real or a complex number, check whether a matrix is diagonal, orthogonal, etc. and much more. **Indices** store uniform types of matrix entry index, row and column slices. This allows to easily change the required type if needed, and the changes will take place throughout the whole library.

```
1 LinAlgTools::Matrix<long double> matrix = {{1e-10, 1e-10}};  
2 LinAlgTools::Matrix<long double> zero_matrix = {{0, 0}};  
3 std::cout << LinAlgTools::AreEqualMatrices(matrix, zero_matrix); //true
```

- **Matrix_traits** and **is_complex** are used to handle different matrix types and complex number respectively. Concepts, mentioned earlier, were introduced in c++ and are a tool which allows to have uniform template for any matrix class.

```
1 template<Core::MatrixType F, Core::MatrixType S>  
2 Matrix<typename F::ElementType> operator+(const F& lhs, const S& rhs) {  
3     \*Implementation*\br/>4 }
```

4.3 Algorithms

This part of the library contains the implementation of the algorithms, which were extensively discussed above. Majority of algorithms' subroutines are implemented as separate functions and can be used as linear transformations. Moreover, the implementation of the algorithms exactly matches its pseudocode counterpart for preserving cohesion and level of abstraction.

- **Householder** contains **HouseholderVectorReduction** for transforming a vector, zeroing out each elements below the first first one. **HouseholderLeftReflection** and **HouseholderRightReflection**, provided a vector, are used to apply a reflection to a matrix.
- **Givens** contains **GetGivensPair**, which calculates coefficients for further rotation. **GivensLeftRotation** and **GivensRightRotation**, provided a vector, are used to apply a rotation to a matrix.
- **QR_Decomposition** contains two implementations of the decomposition: one based on Householder reflections and another based on Givens rotations. The former utilizes **HouseholderVectorReduction** and **HouseholderLeftReflection** to transform the matrix to the required form. Reflections are calculated in-place and to a particular submatrix to avoid unnecessary computations and copies. The latter uses **GetGivensCoefficients**, **GivensLeftRotation** and **GivensRightRotation** to calculate matrices Q and R . Left rotations are applied to submatrices of R , while right rotations are applied to submatrices of Q .

```
1 LinAlgTools::Matrix<int> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
2 auto [Q_1, R_1] = LinAlgTools::Algorithm::HouseholderQR(matrix);  
3 auto [Q_2, R_2] = LinAlgTools::Algorithm::GivensQR(matrix);
```

5 Testing

The library utilizes a testing framework to validate correctness and measure performance. The test architecture is organized as follows:

```
LinAlgTools/  
├── tests/  
│   ├── benchmark_utils/  
│   │   ├── algorithm_benchmark.h  
│   │   └── random_generator.h  
│   ├── const_sub_matrix_tests.cpp  
│   ├── matrix_tests.cpp  
│   ├── qr_performance_tests.cpp  
│   ├── qr_tests.cpp  
│   └── sub_matrix_tests.cpp
```

Figure 3: Directory structure of the **Tests** section

Each implemented function, class and algorithm in the library is tested on multiple test-cases. Proper functioning of the library is ensured via Googletest [2].

Both *QR Decomposition* algorithms and *SVD Decomposition* are benchmarked on random matrices.

6 Conclusion

This section of the paper will contain the results of the development of the programming project with a brief overview of the accomplished work.

References

- [1] Charles F. Van Loan Gene H. Golub. Matrix computations, 4th edition. URL: <https://disk.yandex.ru/i/11ZWE2ud-IZtMA>, 2013.
- [2] Google. Googletest. URL: <https://github.com/google/googletest>, 1.15.2.
- [3] Zinkin Zakhar. LinAlgTools. URL: <https://github.com/Avgustineiw/LinAlgTools>.